🟢 Aula 5 — Computed Properties e Watchers (4h)

🎯 Objetivos da aula

- Diferenciar **computed properties** e **methods**.
- Entender **watchers** e quando usá-los corretamente.
- Implementar **cálculos automáticos** baseados em estado.
- Monitorar mudanças reativas (inputs, listas, flags).
- **Refatorar** o código das aulas anteriores para **boas práticas** (menos lógica no template, mais código declarativo).

---

1) Teoria (aprox. 1h30)

1.1 Computed Properties vs Methods

❌ Methods

- Executados **toda vez** que o template renderiza.
- Não são cacheados.
- Bons para **ações** (eventos).

```
methods: { totalUsuarios() { return this.usuarios.length } }
```

⚠️ Problema: recalcula sempre, mesmo sem mudança.

✅ Computed Properties

- **Cacheadas** com base nas dependências reativas.
- Recalculam **somente quando o estado muda**.
- Ideais para **valores derivados**.

```
computed: { totalUsuarios() { return this.usuarios.length } }
```

✔️ Mais performático
✔️ Código mais limpo
✔️ Padrão de mercado

1.2 Watchers e Reatividade

Watchers observam mudanças em dados específicos.

Usar watcher quando:

- Precisa **reagir** a uma mudança (efeito colateral).
- Chamar API ao mudar valor.
- Validar dados dinamicamente.
- Sincronizar estados.

```
watch: { nome(novoValor) { console.log('Nome mudou:', novoValor) } }
```

⚠️ Não usar watcher para:

- Cálculos derivados simples → use **computed**.

1.3 Regra de Ouro

Situação	Use
Valor derivado	`computed`
Ação / clique	`methods`

Situação	Use
Efeito colateral	`watch`

---

2) Prática (aprox. 2h30) — Refatoração com Computed e Watch

> 🎯 **Contexto**
> Refatorar o sistema de usuários para:

- Mostrar **contadores automáticos**
- Reagir a mudanças no formulário
- Remover lógica desnecessária dos métodos

2.1 `App.vue` — Código refatorado (com boas práticas)

```
<template> <main class="container"> <h1>ILP3 – Vue.js Aula 5</h1> <!-- Dados derivados via computed --> <p> Total de usuários: <strong>{{ totalUsuarios }}</strong> | Ativos: <strong>{{ usuariosAtivos }}</strong> | Inativos: <strong>{{ usuariosInativos }} </strong> </p> <UserForm @salvar="adicionarUsuario" :modo-edicao="modoEdicao" :usuario-edicao="usuarioEdicao" /> <UserList :usuarios="usuarios" @alternar-status="alternarStatus" @remover="removerUsuario" @editar="editarUsuario" /> </main> </template> <script> import UserForm from './components/UserForm.vue' import UserList from './components/UserList.vue' export default { name: 'App', components: { UserForm, UserList }, data() { return { usuarios: [], proximoId: 1, // Controle de edição modoEdicao: false, usuarioEdicao: null } }, /* ====================== COMPUTED PROPERTIES ====================== */ computed: { totalUsuarios() { return this.usuarios.length }, usuariosAtivos() { return this.usuarios.filter(u => u.ativo).length }, usuariosInativos() { return this.usuarios.filter(u => !u.ativo).length } }, methods: { adicionarUsuario(dados) { // Atualiza ou cria usuário if (this.modoEdicao && this.usuarioEdicao) { Object.assign(this.usuarioEdicao, dados) this.modoEdicao = false this.usuarioEdicao = null return } this.usuarios.push({ id: this.proximoId++, ...dados }) }, alternarStatus(id) { const usuario = this.usuarios.find(u
```

```
=> u.id === id) if (usuario) usuario.ativo = !usuario.ativo }, removerUsuario(id) { this.usuarios = this.usuarios.filter(u =>
u.id !== id) }, editarUsuario(usuario) { this.modoEdicao = true this.usuarioEdicao = usuario } } } </script>
```

2.2 `UserForm.vue` — Watcher em ação (monitoramento)

```
<template> <section class="card"> <h2>{{ modoEdicao ? 'Editar Usuário' : 'Novo Usuário' }}</h2> <input v-model="nome"
placeholder="Nome" /> <input v-model="email" placeholder="Email" /> <button @click="salvar"> {{ modoEdicao ? 'Atualizar' :
'Salvar' }} </button> <!-- Feedback reativo --> <p v-if="mensagem" class="info">{{ mensagem }}</p> </section> </template>
<script> export default { name: 'UserForm', props: { modoEdicao: Boolean, usuarioEdicao: Object }, data() { return { nome: '',
email: '', mensagem: '' } }, /* ====================== WATCHERS ====================== */ watch: { // Observa mudança no usuário
em edição usuarioEdicao: { immediate: true, handler(usuario) { if (usuario) { this.nome = usuario.nome this.email = usuario.email
} } }, // Observa o campo nome nome(novoValor) { this.mensagem = novoValor.length < 3 ? 'Nome muito curto' : '' } }, methods: {
salvar() { this.$emit('salvar', { nome: this.nome, email: this.email, ativo: true }) this.nome = '' this.email = '' this.mensagem
= '' } } } </script> <style scoped> .info { margin-top: 10px; color: #f57c00; } </style>
```

---

3) O que foi aplicado (ligando teoria à prática)

✔️ Computed Properties

```
usuariosAtivos() { return this.usuarios.filter(u => u.ativo).length }
```

➡️ Cálculo automático, sem métodos extras.

✔️ Watchers

```
watch: { nome(valor) { if (valor.length < 3) ... } }
```

➡️ Reação a mudanças (efeito colateral).

✔️ Refatoração para boas práticas

- ❌ lógica no template
- ❌ métodos para cálculo
- ✅ computed para derivados
- ✅ watch para efeitos

---

4) Atividade prática da aula

🎯 Desafio

1. Criar um **computed** chamado `usuariosOrdenados`
   - Ordenar por nome (A–Z)
2. Usar `v-for` baseado nesse computed
3. Criar um **watch** que:
   - Mostre mensagem quando total de usuários ultrapassar 5

💡 Dica:

```
watch: { usuarios(novaLista) { if (novaLista.length > 5) { alert('Muitos usuários cadastrados!') } } }
```

---

5) Checklist de encerramento

- Usou computed corretamente
- Evitou métodos para cálculos
- Aplicou watch para efeitos
- Refatorou código legado
- Código mais limpo e declarativo